



From LTL to MLTL: Exploring translation between temporal logic representations using FRET and WEST

Songyan Lai Rosemary Monahan

Department of Computer Science, Maynooth University, Ireland

FTfJP 2025, 3rd of July 2025

Overview

- ▶ Formal Requirements Elicitation.
- ▶ Tool integration with FRET
- ▶ Translation framework to link FRET and WEST tools
- ▶ Towards more tool support for Hetrogenous Verification.

Paths to Hetrogenous Verification

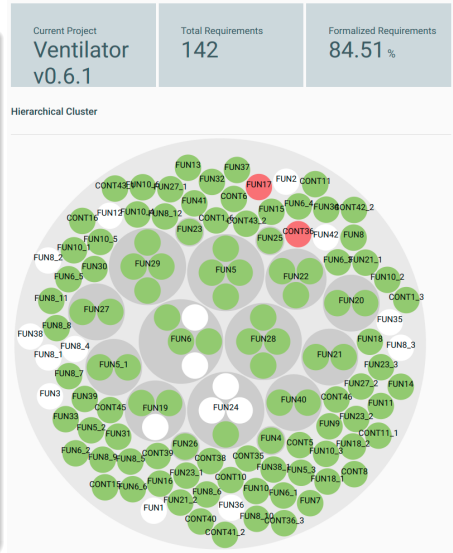
- ▶ Formal Requirements Elicitation via FRET
- ▶ Modelling in Event-B
- ▶ Deductive Verification with Dafny
- ▶ Model Checking with Kind2/NuSMV
- ▶ Runtime Monitoring
- ▶ Workflows and Toolchains for Verification
- ▶ Heterogeneous Verification

FRET

The Formal Requirements Elicitation Tool (FRET)

FRET

- ▶ A tool developed by NASA and extended by Maynooth University's MU-FRET team, designed to translate natural language requirements into formal specifications.
- ▶ Uses "FRETish" – a structured English dialect to systematically map natural language clauses (e.g., conditions, components, timing) into formal Temporal Logic.
- ▶ Automatically generates both Past-Time LTL and Future-Time LTL formulas from FRETish inputs, supporting verification via tools like Kind2 model checker and Copilot runtime monitors.



Overview of our work in VALU3S Project

- ▶ Use Case 5 from VALU3S Project^a
 - ▶ Aircraft engine's software controller provided by Collins Aerospace
- ▶ Formalising requirements in NASA's Formal Requirements Elicitation Tool (FRET)^b
 - ▶ Requirements Elicitation
 - ▶ Creating parent-child requirements
- ▶ Support for refactoring of requirements in Mu-FRET^c
- ▶ Modelling in Event-B^d
- ▶ Toolchain to support model checking of formalised requirements
- ▶ <https://repo.valu3s.eu/>

^aVALU3S: <https://valu3s.eu>

^bFRET: <https://github.com/NASA-SW-VnV/fret>

^cMu-FRET: <https://github.com/valu3s-mu/mu-fret>

^dEvent-B: <http://www.event-b.org/>

Use case 5:

<https://repo.valu3s.eu/use-cases/aircraft-engine-controller>

- ▶ 1 Simulink diagram
- ▶ 14 Natural-language requirements
- ▶ 20 Test cases
- ▶ 4 Evaluation scenarios

Requirements Elicitation

Analysis: Aircraft Engine Controller Requirements

- ▶ 14 natural-language requirements.

ID	Description
UC5_R_1	Under sensor faults, while tracking pilot commands, control objectives shall be satisfied (e.g., settling time, overshoot, and steady state error will be within predefined, acceptable limits)
UC5_R_2	Under sensor faults, during regulation of nominal system operation (no change in pilot input), control objectives shall be satisfied (e.g., settling time, overshoot, and steady state error will be within predefined, acceptable limits)
UC5_R_3	Under sensor faults, while tracking pilot commands, operating limit objectives shall be satisfied (e.g., respecting upper limit in shaft speed)
UC5_R_4	Under sensor faults, during regulation of nominal system operation (no change in pilot input), operating limit objectives shall be satisfied (e.g., respecting upper limit in shaft speed)

Farrell, M., Luckcuck, M., Sheridan, O., Monahan, R. FRETting about Requirements: Formalised Requirements for an Aircraft Engine Controller. *International Conference on Requirements Engineering: Foundation for Software Quality*. pp. 96–111. Springer (2022).

FRETish Example: Requirement 1

- ▶ Natural-Language Requirement 1: *“Under sensor faults, while tracking pilot commands, control objectives shall be satisfied (e.g. settling time, overshoot, and steady state error will be within predefined, acceptable limits)”*
- ▶ In FRETish: `if sensorfaults & trackingPilotCommands Controller shall satisfy controlObjectives`

Formal Requirements Elicitation Tool: FRET

- ▶ supports the formalisation, understanding and analysis of requirements user-friendly interface
- ▶ intuitive diagrammatic explanations of requirement semantics
- ▶ users specify requirements in restricted natural language, called FRETish, which embodies a temporal logic semantics

SCOPE

CONDITIONS

COMPONENT*

SHALL*

TIMING

RESPONSES*

Using FRET

```
UC5_R_1: if sensorfaults &  
trackingPilotCommands Controller  
shall satisfy controlObjectives
```

Future Time LTL



SMV  ☒ Infinite trace

Format

```
((G (((! (sensorfaults & trackingPilotCommands)) & (X  
(sensorfaults & trackingPilotCommands))) -> (X (F  
controlObjectives)))) & ((sensorfaults &  
trackingPilotCommands) -> (F controlObjectives)))
```

Target: ControlledSystem component.

CoCoSim: Contract based **C**ompositional verification of **S**imulink models.

- ▶ FRET can generate CoCoSpec assume-guarantee contracts for Simulink blocks.
- ▶ CoCoSpec contracts are added to the Simulink diagram.
- ▶ Contracts are checked during simulations of the diagram using the Kind2 model checker.

CoPilot: Runtime monitoring framework.

LTL semantics: also used to generate runtime monitors for the implemented system.

FRET requirements without timing constraints: used to specify requirements in other formalisms e.g. Event-B, Dafny.

WEST

WEST: Validating Mission-Critical Timelines

- ▶ WEST (acronym from authors' first names: Zili Wang, Jenna Elwing, Jeremy Sorkin, Chiara Traveset) – open-source tool for validating Mission-time Linear Temporal Logic (MLTL) formulas.
- ▶ Automatically generates regular expressions representing all satisfying computations/timelines of MLTL formulas (describing valuations/truth table rows that satisfy the formula).
- ▶ Workflow involves 3-step algorithm:
 - ▶ (1) parse MLTL into Abstract Syntax Tree (AST)
 - ▶ (2) convert AST to Non-deterministic Finite Automaton (NFA)
 - ▶ (3) generate regular expressions from NFA.
- ▶ Includes a GUI for visualizing generated regular expressions, along with development-testing scripts and datasets.

WEST: Validating Mission-Critical Timelines

WEST MLTL Formula Validation Tool

1 $((p0 \ \& \ !(F[0,3] \ !p1)) \rightarrow p2)$ Run Grammar 9 MLTL Formula: $((p0 \ \& \ G[0,3] \ p1) \rightarrow p2)$

☐ Optimize Bits 2 ☐ Apply REST 3

8 trace: 010,000,011,010 Help

Formula: $((p0 \ \& \ G[0,3] \ p1) \rightarrow p2)$ 4

Import trace 010,000,011,010 11

Export trace trace.csv 12

Unexpected Formula?

Please select a subformula to explore:

- 5 $((p0 \ \& \ G[0,3] \ p1) \rightarrow p2)$
- $(p0 \ \& \ G[0,3] \ p1)$
- $G[0,3] \ p1$
- $p2$
- $p0$
- $p1$

14 Rand SAT 15 Rand UNSAT

13 reset 0 1 2 3

p0	☐	☐	☐	☐
p1	☒	☐	☒	☒
p2	☐	☐	☒	☐

6 Regexp List Backbone Analysis 7

0ss,sss,sss,sss 16

s0s,sss,sss,sss

sss,s0s,sss,sss

sss,sss,s0s,sss

sss,sss,sss,s0s

WEST MLTL Formula Validation Tool

10 MLTL Formula: $((p0 \ \& \ G[0,3] \ p1) \rightarrow p2)$

trace: 110,010,011,010 Help

Import trace 110,010,011,010

Export trace trace.csv

Rand SAT Rand UNSAT

reset 0 1 2 3

p0	☒	☐	☐	☐
p1	☒	☒	☒	☒
p2	☐	☐	☒	☐

Regexp List Backbone Analysis 17

Backbone for SAT Assignments:

t = 0:

t = 1:

t = 2:

t = 3:

Backbone for UNSAT Assignments:

t = 0: p0, p1, ~p2

t = 1: p1

Addressing Research Gaps

Existing Integrations of Requirement Formalization Tools

FRET: Integration with Verification Ecosystem

Integrated Tool	Integration Purpose	Verification Outcome
nuXmv	Export LTL formulas for model checking	Validates unbounded temporal properties
CoCoSim	Generate CoCoSpec assume-guarantee contracts for Simulink models	Checks contracts via Kind2 during Simulink simulations
CoPilot	Compile FRET-generated monitors into real-time C code	Deploys hard real-time monitors on embedded systems

WEST: Limited Practical Integration

- ▶ Applied in *R2U2* for UAV runtime monitoring (WEST-validated MLTL formulas).
- ▶ Manual workflows are error-prone and time-intensive.

Research Gaps: Syntax Mismatch & Semantic Challenges

1. Syntax Mismatch

- ▶ Unbounded (FRET) vs. bounded intervals (WEST)
- ▶ Descriptive names generic IDs (traceability risk)
- ▶ Nested operators interval/semantics conflicts

2. Semantic Gaps

- ▶ Finite-trace constructs (e.g., LAST V, persisted) lack direct MLTL equivalents
- ▶ FRET's time-quantified operators and time-quantified operators conflict with MLTL's future-oriented syntax.

3. Traceability Loss

- ▶ **Bidirectional Gap:** WEST's validation results (e.g., conflicting timelines) cannot auto-translate to FRET requirements.
- ▶ **Manual Overhead:** Longer manual cross-checks are required for audits.

4. Feedback Loop Deficiency

- ▶ *Mission-Time Feedback:* FRET-nuXmv lacks mission-time loops.
- ▶ *Test Case Gap:* Early unavailability of WEST-compatible test cases.

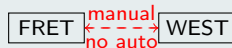


Figure 1: Disjoint Toolchain Schematic

Buiding the translator

► Regex-based Transformation

- Suited for linear, templated FRET output with shallow nesting (3 layers)
- Pattern matching accuracy >95% with $O(n)$ complexity—critical for real-time/critical systems

► Auditability

- Incremental transformation logs (e.g., `steps.push(...)`) for traceability from FRETish origin to MLTL
- NASA-grade audit trails linking outputs to source specifications

► Iterative Phases

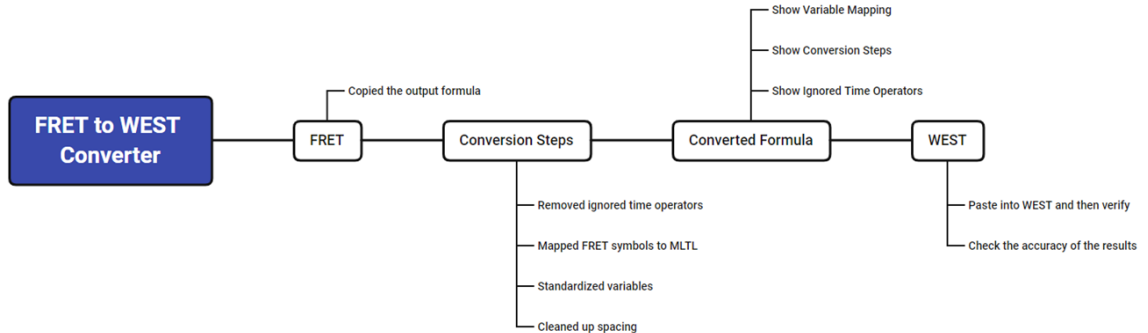
- Variable Standardization ($v1 \rightarrow v3$):
sequential numbering to resolve syntax conflicts (e.g., `!StartupMode!p1`) Context – Aware Mapping (*recursive parsing for cross – formula consistency*)

► Modular Architecture (v7v9): separate functions (e.g., `mapSymbols`, `standardizeVariables`, `cleanUpSpacing`, `removeRedundantParentheses`, `detectExtraParentheses`) for readability, maintenance, immediate error feedback

- Code versions available in repository; refined via aerospace medical case studies

The conversion process during the test phrase

shows the conversion process.png



► Front-End Simulation

- Minimal HTML5/JavaScript UI: input FRET formula, convert, review ignored operators, variable mappings, final MLTL output
- Allows early testing without full WEST test suite

► Iterative Testing

- Use MU-FRET test cases; gradually add WEST-side cases via collaboration
- Validation loop: convert simulate MLTL check refine translation rules

► Audit Feedback

- Detailed conversion logs for debugging and certification
- Collaboration with MU-FRET and WEST teams for full pipeline integration

Graphic User Interface

≡

MU-FRET

Projects ▾

CREATE

⌵

⌵

⌵

⌵

?

i

FUN8_5   

Ventilator v0.6.1

RATIONALE
The system shall not permit the healthcare professional operator to erase the contents of the alarm system log.

REQUIREMENT
System shall always satisfy if user = operator then !eraseLog.

SEMANTIC DESCRIPTION
ENFORCED: in the interval defined by the entire execution. TRIGGER: first point in the interval. REQUIRES: for every trigger, RES must hold at all time points between (and including) the trigger and the end of the interval.

beginning of Time



SEMANTIC DIAGRAM
Response = $(user = operator \Rightarrow !eraseLog)$.

FUTURE TIME FORMULA
 $(LAST \ \forall \ ((user = operator) \rightarrow (!eraseLog)))$

PAST TIME FORMULA
 $(H \ ((user = operator) \rightarrow (!eraseLog)))$

CONVERT TO WEST

CLOSE

Graphic User Interface

The screenshot shows a software window titled "FUN8_5" with a toolbar containing a pencil, a wrench, and a trash icon. The window is labeled "Ventilator v0.6.1" in the top right corner. On the left side, there is a vertical sidebar with icons for a window, a list, navigation arrows, a downward arrow, an upward arrow, a question mark, and an information icon. The main content area displays the following information:

SEMANTIC DIAGRAM
Response = $(user = operator \Rightarrow !eraseLog)$.

FUTURE TIME FORMULA
 $(LAST\ V\ ((user = operator) \rightarrow (!eraseLog)))$

PAST TIME FORMULA
 $(H\ ((user = operator) \rightarrow (!eraseLog)))$

CONVERT TO WEST

MTL OUTPUT
 $((p1=p2) \rightarrow (p3))$

VARIABLE MAPPING
p1: user
p2: operator
p3: eraseLog

IGNORED TIME OPERATORS
LAST V: Indicates current state relevance; removed in conversion.

CONVERSION STEPS
Original formula: $(LAST\ V\ ((user = operator) \rightarrow (!eraseLog)))$
Removed ignored time operators: $((user = operator) \rightarrow (!eraseLog))$
Mapped FRET symbols to MTL: $((user = operator) \rightarrow (!eraseLog))$
Standardized variables: $((p1 = p2) \rightarrow (!p3))$
Cleaned up spacing: $((p1=p2) \rightarrow (p3))$
Final converted formula: $((p1=p2) \rightarrow (p3))$

Evaluation

Case Study: Access Control

FRET MLTL Translation

- ▶ **FRET Input** “ System shall always satisfy if user = operator then !eraseLog ”
- ▶ **FRET Output:** “ (LAST $\vee$ ((user = operator) $\rightarrow$ (! eraseLog)) ”
- ▶ **Translated MLTL:** “ ((p1=p2) $\rightarrow$ (!p3)) ”
- ▶ Semantic preserved: implication negation maintained
- ▶ Semantic loss: LAST $\vee$ (terminal constraint) omitted; descriptive names hidden

Adaptive Logging

- ▶ Omitted constructs flagged in logs for review
- ▶ Mapping metadata exported (e.g., puser) for traceability

► Scalability Constraints:

- Deep nesting (>15 operators) causes state-space explosion
- Artificial bounds for unbounded operators (e.g., $G[0,1000]$) can over-constrain (12%)
- Mitigation: cloud deployment for large mission phases

► Variable Syntax Rigidity:

- pN-style variables hide semantics; initial audit misalignment (15%)
- Collaborative bidirectional mapping table for semantic recovery

► Interoperability Issues:

- Syntax translation distortions; need formal equivalence checking
- Planned integration of SMT solvers (e.g., Z3) for semantic validation

► Human-Centered Design:

- Engineers struggled with pN; visualization of mappings improved comprehension
- Dynamic tooltips in FRET interface to show original names

Tool support for Hetrogenous Verification

Conclusions & Future Work

Key Takeaways

- ▶ **Automated translator**: regex-based LTLMLTL with trace emulation
- ▶ **Semantic heuristics**: maintain equivalence amid interval/nesting conflicts
- ▶ **Traceability**: audit logs link MLTL results back to FRET specs
- ▶ **Validation**: 85% syntactic correctness, 80% semantic similarity

Future Directions

- ▶ SMT integration for formal equivalence checking
- ▶ Runtime monitors probabilistic checkers for redundancy
- ▶ Cloud-native WEST deployment for large-scale verification
- ▶ AI-driven annotation mapping errors back to FRET specs
- ▶ Long-term: re-architect WEST (TypeScript/WebAssembly, GPU acceleration, debugger)

- ▶ Natural-Language Requirement Tracability
 - ▶ Difficult to express logically for formal methods.
 - ▶ Tools like FRET help.
 - ▶ Maintaining requirements is error-prone (repetition and dependencies).
- ▶ Our Experience: Aircraft Engine Controller, MLV, Grasping, Drones...
 - ▶ Formalise requirements in FRETish.
 - ▶ Build **traceability** from requirements to verification conditions.
 - ▶ Refactor to improve **maintainence** of requirements.
 - ▶ Provide **toolchains and information exchange** for verification.
 - ▶ **Heterogeneous/corroborative verification** is the way forward
- ▶ Aiming to improve toolchain support from FRET to other tools

Acknowledgments

- ▶ **NASA FRET** and **WEST** teams for foundational tool development.
- ▶ **Oisín Sheridan** (Maynooth University) and **Zili Wang** (Iowa State University) for technical guidance and support throughout the integration.

*“The purpose of abstraction is not to be vague,
but to create a new semantic level in which one can be absolutely precise.”*
— Edsger W. Dijkstra

Related Publications

- ▶ Sheridan, O., Becker, L.B., Farrell, M., Luckcuck, M., Monahan, R. Sharper Specs for Smarter Drones: Formalising Requirements with FRET. *In: Hess, A., Susi, A. (eds) Requirements Engineering: Foundation for Software Quality. REFSQ 2025*
- ▶ Farrell, M., Luckcuck, M., Sheridan, O., Monahan, R. Adventures in FRET and Specification *In: Bridging gaps between program specification paradigms (Specify This), ISoLA 2024*
- ▶ Farrell, M., Luckcuck, M., Sheridan, O., Monahan, R. FRETting and Formal Modelling: A Mechanical Lung Ventilator *Rigorous State-Based Methods. ABZ 2024*
- ▶ Luckcuck, M., Farrell, M., Sheridan, O., Why just FRET when you can Refactor? Retuning FRETish Requirements. *arXiv preprint arXiv:2202.05816. 2022*
- ▶ Luckcuck, M., Farrell, M., Sheridan, O. A Methodology for Developing a Verifiable Aircraft Engine Controller from Formal Requirement, *IEEE Aerospace Conference 2022.*
- ▶ Luckcuck, M., Farrell, M., Sheridan, O. Towards Refactoring FRETish Requirements *NASA Formal Methods 2022.*
- ▶ Farrell, M., Luckcuck, M., Sheridan, O., Monahan, R. FRETting about Requirements: Formalised Requirements for an Aircraft Engine Controller. *International Conference on Requirements Engineering: Foundation for Software Quality 2022.*
- ▶ Bukhari, SAA., Flinkow, T., Inkarbekov, M., Pearlmutter, BA., Monahan, R. Creating a Formally Verified Neural Network for Autonomous Navigation: An Experience Report. *Formal Methods for Autonomous Systems 2024.*
- ▶ Flinkow, T., Pearlmutter, BA., Monahan, R. Comparing Differentiable Logics for Learning with Logical Constraints *Formal Methods for Autonomous Systems 2023 and Sci of Comp Prog 2024*
- ▶ Flinkow, T., Pearlmutter, BA., Monahan, R. Immersive Neural Network Exploration: A VR Approach to Human-Centered AI Understanding. *Conference on Human Centered Artificial Intelligence (HCAI-ep) 2023.*

Acknowledgements

- ▶ Marie Farrell and Conor Reynolds, University of Manchester
- ▶ Matt Luckcuck, University of Nottingham
- ▶ Oisin Sheridan, Medet Inkarbekov, Ali Bukhari, Thomas Flinkow, Barak Pearlmutter - Maynooth University
- ▶ Georgios Giantamidis, Stylianos Basagiannis and Vassilios A. Tsachouridis - United Technologies Research Center, Ireland
- ▶ Anastasia Mavridou - KBR/NASA Ames Research Center, USA
- ▶ Funding:
 - ▶ Science Foundation Ireland SFI/20/FFP-P/8853 and ADAPT FI
 - ▶ VALU3S project (grant No 876852)
 - ▶ Enterprise Ireland (grant No IR20200054)
 - ▶ MU Hume Scholarship

► Isomorphic Semantic Mapping

- Bijective mapping $\mu : V_{\text{FRET}} \leftrightarrow V_{\text{MLTL}}$, preserving semantics and structure
- Ensures behavioral equivalence, e.g.
 $\text{engine_overheat} \rightarrow \text{shutdown_protocol} \equiv p_1 \rightarrow p_2$

► Namespace Partitioning

- Disjoint proposition spaces: $V_{\text{FRET}} \cap V_{\text{MLTL}} = \emptyset$
- Unique IDs for FRET variables $(p_1, \dots, p_N)$ to avoid collisions

► Temporal Context Conservation

- Time-indexed equivalence: behaviors aligned over mission timeline
- Finite-trace projection: prefix of infinite trace in MLTL preserves FRET prefix semantics

► Version Stability

- Monotonic consistency across versions: $\mu(V_k) \subseteq \mu(V_{k+1})$
- Backward compatibility via partial mapping ensuring $\pi \circ \mu(\varphi_{k+1}) \models \mu(\varphi_k)$

► Theoretical Limits

- Natural-language labels omitted in core translation; referenced separately for engineers



VALUS



Taighde Éireann
Research Ireland

Funding:



Further Information and Resources: <http://www.cs.nuim.ie/research/pop/>